FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	Keisha Belle C. Calibjo Jose Gabriel M. Valencia
Section:	AN43
Professor:	Crisola G. Tan
Date:	11/02/26

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.

7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)

2. Software

A programming environment:
   - Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
   - Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:
   - MS Word, Google Docs, or LibreOffice
   - Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:
   - Knowledge Representation & Reasoning
   - Case-Based Reasoning cycle
   - Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
   - Google Drive for shared documentation
   - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases.

Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✔ Similarity score or ranking

✔ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.

This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✔ "New case saved" or similar confirmation

✔ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

- For this project, Calibjo mainly focused on the documentation and front end, while Valencia focused mostly on the back end that stores disaster cases, compares them, and adapts plans. Calibjo helped explain how the system works and designed the dashboard so users can enter a disaster scenario, review similar cases, and understand the results. One challenge was deciding how to compare scenarios with different details, such as severity, affected population, road access, and available resources. We also had to make the recommendations clear and explain that a similar past case can guide a plan but cannot guarantee it will work in a real disaster. Through this project, we learned that Case-Based Reasoning (CBR) solves new problems by retrieving a relevant past case, reusing and adapting its solution, revising it, and retaining reviewed experience for future use. We also learned that the usefulness of CBR depends on the quality of its cases and how carefully proposed solutions are reviewed.

Case Studies:

Case Study 1: Disaster Response Decision Support System
A CBR-based DSS helps local government units respond to natural disasters. • Past cases include typhoons, floods, and earthquakes. • New event: *Typhoon with expected flooding in urban areas.* • System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*. • It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.

- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:
CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:
Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft
- digital fraud
- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.

Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.

Learning:
System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

main.py code

```
"""Open the dashboard with python main.py; use --cli for the console."""

from __future__ import annotations

import argparse
import json
import tempfile
from dataclasses import asdict
from pathlib import Path

from cbr import (
    AREA_TYPES, DISASTER_TYPES, MIN_SIMILARITY, PRIORITIES, ROAD_ACCESS,
    Adaptation, Case, CaseStore, Match, Problem, Solution, adapt, retrieve, revise,
)

ROOT = Path(__file__).resolve().parent
```

```
SEED = ROOT / "data" / "seed_cases.json"


class Cancelled(Exception):
    pass


def answer(prompt: str) -> str:
    value = input(prompt).strip()
    if value.lower() == ":back":
        raise Cancelled
    return value


def number(label: str, minimum: int, maximum: int, default: int | None = None) -> int:
    while True:
        suffix = f" [{default}]" if default is not None else ""
        value = answer(f"{label}{suffix}: ")
        if not value and default is not None:
            return default
        try:
            result = int(value)
            if minimum <= result <= maximum:
                return result
        except ValueError:
            pass
        print(f"Enter a whole number from {minimum:,} to {maximum:,}.")


def choice(label: str, options: tuple[str, ...], default: str | None = None) -> str:
    print(f"\n{label}: " + " | ".join(f"{i}. {item}" for i, item in enumerate(options, 1)))
    selected = number("Choose", 1, len(options), options.index(default) + 1 if default else None)
    return options[selected - 1]


def text(label: str, default: str | None = None) -> str:
    while True:
        value = answer(f"{label}" + (f" [{default}]" if default else "") + ": ")
        if not value and default:
            return default
        if value and len(value) <= 1000:
            return value
        print("Enter between 1 and 1,000 characters.")
```

```
def confirm(label: str) -> bool:
    while True:
        value = answer(label + " [y/N]: ").lower()
        if value in ("y", "yes"):
            return True
        if value in ("", "n", "no"):
            return False
        print("Enter y or n.")


def read_problem() -> Problem:
    print("\n1. CASE REPRESENTATION - describe a new classroom scenario")
    print("Severity is an exercise rating from 1 to 5, not an official warning level.")
    print("Enter :back at any prompt to return to the menu.")
    return Problem(
        disaster_type=choice("Disaster type", DISASTER_TYPES),
        severity=number("Severity (1 = low, 5 = very high)", 1, 5),
        affected_people=number("Affected people", 1, 1_000_000),
        road_access=choice("Road access", ROAD_ACCESS),
        area_type=choice("Area type", AREA_TYPES),
        available_shelter_places=number("Available shelter places (people)", 0, 1_000_000),
        available_response_teams=number("Available response teams", 0, 100_000),
    )


def show_matches(matches: list[Match]) -> None:
    print("\n2. RETRIEVE - reviewed cases with the same disaster type")
    print(f"{'No.':<5}{'Case':<20}{'Similarity':>12}{'Severity':>11}{'People':>12}")
    for i, match in enumerate(matches, 1):

print(f"{i:<5}{match.case.id:<20}{match.score:>11.1%}{match.case.problem.severity:>11}{match.case.probl
em.affected_people:>12,}")
    print("Similarity measures feature resemblance; it is not probability of success.")


def show_reasoning(match: Match, query: Problem) -> None:
    print(f"\nWhy {match.case.id} matched:")
    print(f"{'Feature':<28}{'New':<14}{'Stored':<14}{'Weight':>8}{'Local':>9}{'Contribution':>15}")
    for feature in match.features:
        left = str(getattr(query, feature.feature))
        right = str(getattr(match.case.problem, feature.feature))
```

```
print(f"{feature.feature:<28}{left:<14}{right:<14}{feature.weight:>8.0%}{feature.similarity:>9.1%}{feature.contribution:>15.2%}")
    print(f"Weighted sum = {match.score:.4f} ({match.score:.1%}).")


def show_plan(problem: Problem, plan: Solution, explanation: tuple[str, ...] = ()) -> None:
    print(f"\nIllustrative planning priority: {plan.priority.upper()}")
    print(f"Proposed total shelter places: {plan.shelter_places:,}")
                    print(f"Additional shelter places needed: {max(0, plan.shelter_places - problem.available_shelter_places):,}")
    print(f"Proposed total response teams: {plan.response_teams:,}")
    print(f"Additional teams needed: {max(0, plan.response_teams - problem.available_response_teams):,}")
    for line in explanation:
        print("  Reason: " + line)
    for action in plan.actions:
        print("  Review: " + action)


def reviewed_solution(problem: Problem, proposed: Solution | None = None) -> Solution:
    print("\n4. REVISE - enter your classroom review of the proposed solution")
    if proposed:
        print("Press Enter to keep a proposed value.")
    priority = choice("Planning priority", PRIORITIES, proposed.priority if proposed else None)
      shelters = number("Proposed shelter places", 0, problem.affected_people, proposed.shelter_places if proposed else None)
     teams = number("Proposed response teams", 1, 100_000, proposed.response_teams if proposed else None)
    if proposed:
        actions = proposed.actions
        if confirm("Replace the proposed review actions?"):
            actions = read_actions()
    else:
        actions = read_actions()
    return Solution(priority, shelters, teams, actions)


def read_actions() -> tuple[str, ...]:
    while True:
        value = text("Review actions, separated by semicolons")
        actions = tuple(item.strip() for item in value.split(";") if item.strip())
        if 1 <= len(actions) <= 10 and all(len(item) <= 300 for item in actions):
            return actions
```

```
        print("Provide 1 to 10 actions, each at most 300 characters.")


def retain_review(store: CaseStore, problem: Problem, proposed: Solution | None, origin: str | None) ->
None:
    solution = reviewed_solution(problem, proposed)
    outcome = text("Observed outcome or classroom-review note")
    show_plan(problem, solution)
    if not confirm("Have you reviewed this solution and want to RETAIN it?"):
        print("Not retained. The library was unchanged.")
        return
    case = store.retain(problem, solution, outcome, reviewed=True, adapted_from=origin)
    print(f"\n5. RETAIN - saved {case.id}. Library now contains {len(store.load())} cases.")
    nearest = retrieve(problem, store.load())[0]
    print(f"Retrieving the same problem now returns {nearest.case.id} at {nearest.score:.1%} similarity.")


def solve(store: CaseStore) -> None:
    problem = read_problem()
    matches = retrieve(problem, store.load())
    if not matches:
        print("No reviewed case for this disaster type. Use menu 3 to add one manually.")
        return
    show_matches(matches)
    if matches[0].score < MIN_SIMILARITY:
        print("All matches are below 65%. No automatic adaptation; add a reviewed case manually.")
        return
    selected = number("Choose a case to adapt (0 = cancel)", 0, len(matches), 1)
    if not selected:
        return
    match = matches[selected - 1]
    show_reasoning(match, problem)
    proposed: Adaptation = adapt(problem, match.case)
    print("\n3. REUSE / ADAPT - apply the selected case to the new problem")
    show_plan(problem, proposed.solution, proposed.explanation)
    if any(case.problem == problem for case in store.load()):
        print("This exact problem is already in the library; no duplicate will be added.")
    elif confirm("Revise this proposal and consider retaining a new case?"):
        retain_review(store, problem, proposed.solution, match.case.id)


def browse(store: CaseStore) -> None:
    cases = store.load()
```

```
    print(f"\nCASE LIBRARY - {len(cases)} cases")
    for i, case in enumerate(cases, 1):
        p = case.problem
        print(f"{i:>2}. {case.id:<20} {p.disaster_type:<12} severity {p.severity} | {p.affected_people:,} people |
{case.source}")
    if cases:
        index = number("Inspect case (0 = menu)", 0, len(cases), 0)
        if index:
            print(json.dumps(asdict(cases[index - 1]), indent=2))


def demo() -> None:
    """Demonstrate every stage using a temporary, genuinely written case file."""
    print("\nWORKED EXAMPLE - synthetic flood scenario; temporary case library")
    with tempfile.TemporaryDirectory(prefix="disaster-cbr-demo-") as directory:
        store = CaseStore(Path(directory) / "cases.json", SEED)
        before = len(store.load())
        query = Problem("flood", 5, 1200, "restricted", "urban", 700, 5)
        print("\n1. CASE REPRESENTATION")
        print(json.dumps(asdict(query), indent=2))
        matches = retrieve(query, store.load())
        show_matches(matches)
        best = matches[0]
        show_reasoning(best, query)
        result = adapt(query, best.case)
        print("\n3. REUSE / ADAPT")
        show_plan(query, result.solution, result.explanation)
        print("\n4. REVISE - example reviewer adds one coordination team.")
        corrected = revise(result.solution, result.solution.shelter_places, result.solution.response_teams + 1)
        print(f"Reviewed response teams: {result.solution.response_teams} -> {corrected.response_teams}.")
        learned = store.retain(
            query, corrected,
            "Synthetic classroom review: accepted shelter estimate and added one coordination team.",
            reviewed=True, adapted_from=best.case.id,
        )
        print(f"\n5. RETAIN - wrote {learned.id} to the temporary JSON file.")
        reloaded = store.load()
        nearest = retrieve(query, reloaded)[0]
        print(f"Case count: {before} -> {len(reloaded)}.")
        print(f"New query retrieves learned case: {nearest.case.id == learned.id}.")
        print(f"Similarity after learning: {nearest.score:.1%}.")
        print(f"Learned solution: {nearest.case.solution.response_teams} reviewed response teams.")
    print("Demo complete. Your saved library was not changed.")
```

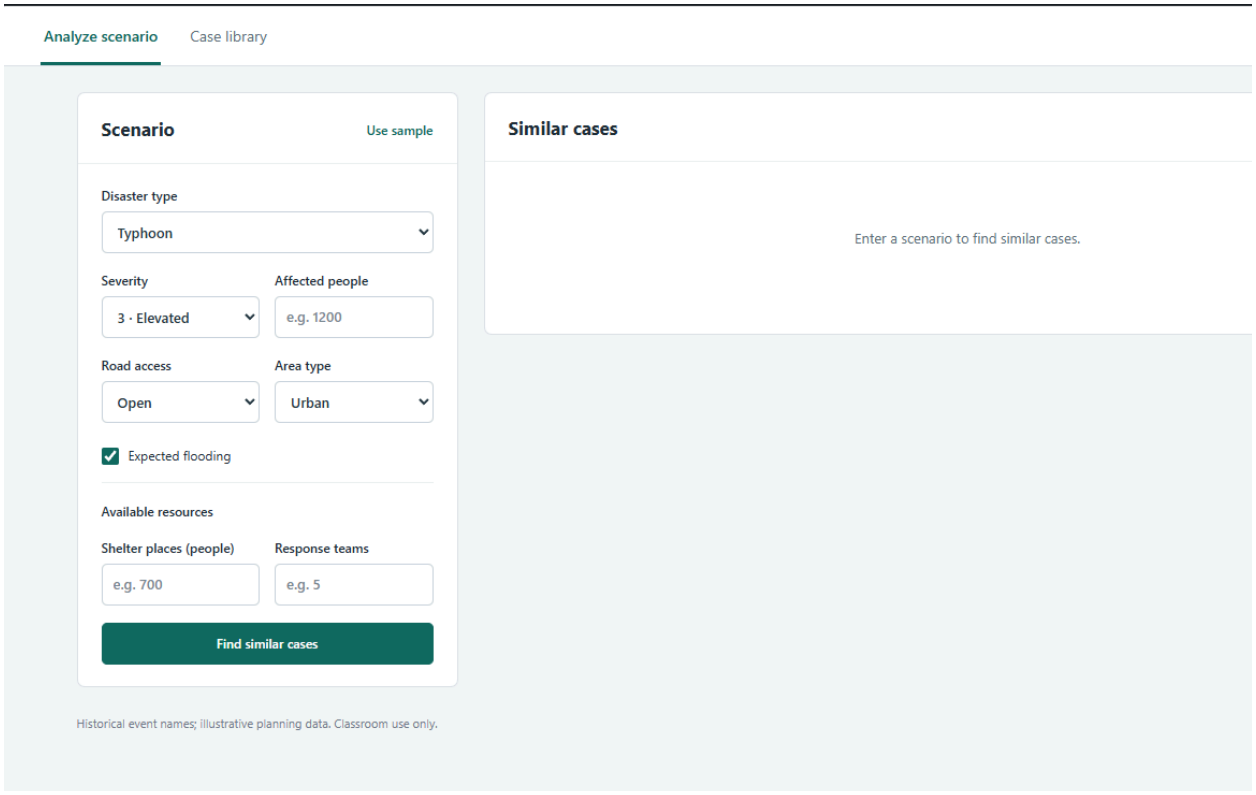
```
def main() -> int:
      parser = argparse.ArgumentParser(description="Disaster DSS: a classroom Case-Based Reasoning program.")
    parser.add_argument("--demo", action="store_true", help="Run all CBR stages without changing saved cases.")
    parser.add_argument("--cli", action="store_true", help="Use the original terminal interface instead of the dashboard.")
   parser.add_argument("--port", type=int, default=0, help="Dashboard port (default: choose a free port).")
    parser.add_argument("--no-browser", action="store_true", help="Start the dashboard without opening a browser.")
    parser.add_argument("--data-file", type=Path, default=ROOT / "data" / "cases.json", help="JSON library for the dashboard or terminal interface.")
   args = parser.parse_args()
   if not args.cli and not args.demo:
      from dashboard import run_dashboard
      return run_dashboard(args.data_file, args.port, not args.no_browser)
   print("DISASTER DSS | Case-Based Reasoning")
   print("Synthetic classroom examples only. Not for real emergency decisions.")
   try:
      if args.demo:
         demo()
         return 0
      store = CaseStore(args.data_file, SEED)
      print(f"Saved library: {store.path}")
      store.load()
      while True:
           print("\n1. Analyze a new scenario\n2. Inspect stored cases\n3. Add a reviewed case manually\n4. Run the worked example\n5. Exit")
         try:
            selected = number("Menu", 1, 5)
            if selected == 1:
               solve(store)
            elif selected == 2:
               browse(store)
            elif selected == 3:
               retain_review(store, read_problem(), None, None)
            elif selected == 4:
               demo()
            else:
               return 0
         except Cancelled:
```

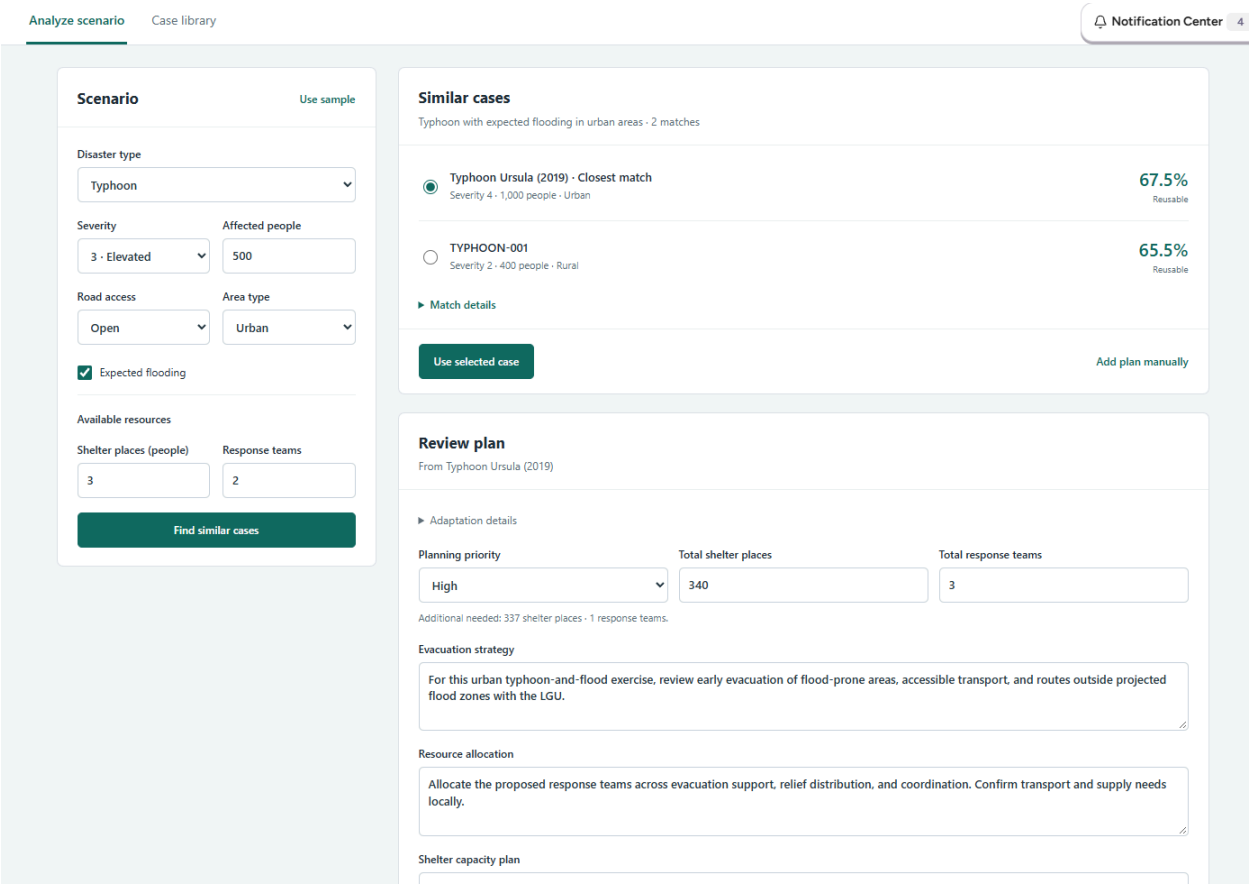
```
            print("Cancelled. No unfinished case was retained.")
        except ValueError as error:
            print(f"Could not complete the operation: {error}")
    except (EOFError, KeyboardInterrupt):
        print("\nGoodbye. No unfinished case was retained.")
        return 0
    except (OSError, ValueError) as error:
        print(f"Error: {error}")
        return 1

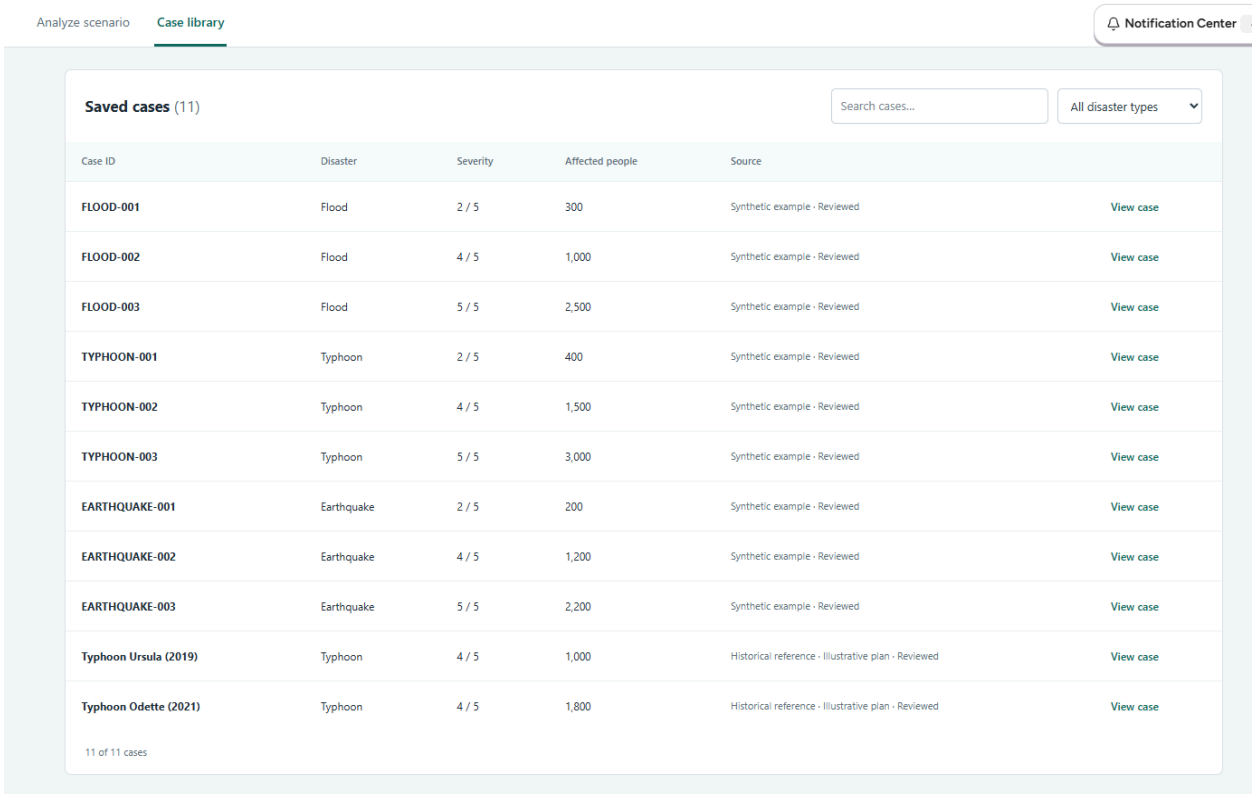

if __name__ == "__main__":
    raise SystemExit(main())
```

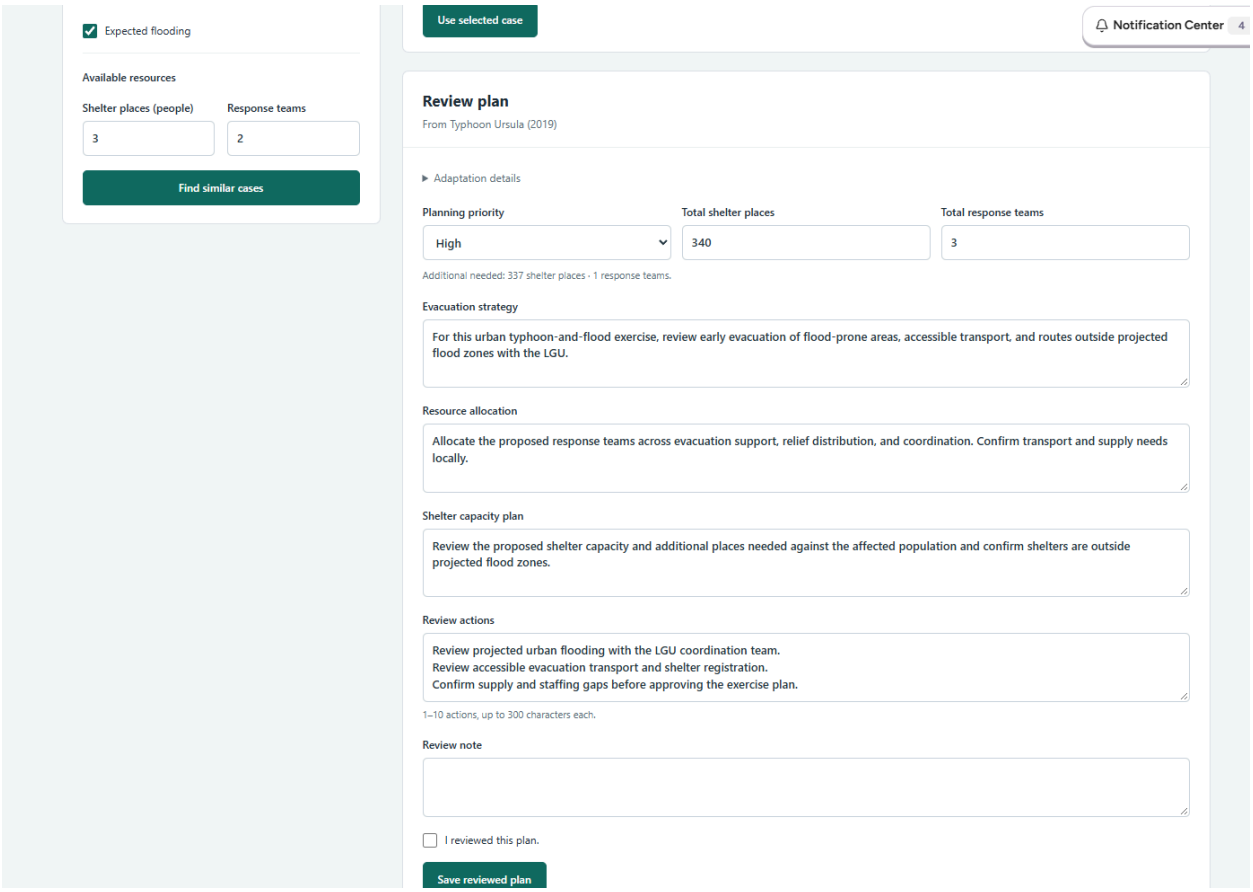
The rest of the code is inside the repository…….

Documentation:




Analyze scenario
Case library
Notification Center 4
Scenario
Use sample
Disaster type
Typhoon
Severity
3 · Elevated
Affected people
500
Road access
Open
Area type
Urban
Expected flooding
Available resources
Shelter places (people)
3
Response teams
2
Find similar cases
Similar cases
Typhoon with expected flooding in urban areas · 2 matches
Typhoon Ursula (2019) · Closest match
Severity 4 · 1,000 people · Urban
67.5%
Reusable
TYPHOON-001
Severity 2 · 400 people · Rural
65.5%
Reusable
Match details
Use selected case
Add plan manually
Review plan
From Typhoon Ursula (2019)
Adaptation details
Planning priority
High
Total shelter places
340
Total response teams
3
Additional needed: 337 shelter places · 1 response teams.
Evacuation strategy
For this urban typhoon-and-flood exercise, review early evacuation of flood-prone areas, accessible transport, and routes outside projected flood zones with the LGU.
Resource allocation
Allocate the proposed response teams across evacuation support, relief distribution, and coordination. Confirm transport and supply needs locally.
Shelter capacity plan


Analyze scenario
Case library
Notification Center
Saved cases (11)
Search cases...
All disaster types
Case ID
Disaster
Severity
Affected people
Source
FLOOD-001
Flood
2 / 5
300
Synthetic example · Reviewed
View case
FLOOD-002
Flood
4 / 5
1,000
Synthetic example · Reviewed
View case
FLOOD-003
Flood
5 / 5
2,500
Synthetic example · Reviewed
View case
TYPHOON-001
Typhoon
2 / 5
400
Synthetic example · Reviewed
View case
TYPHOON-002
Typhoon
4 / 5
1,500
Synthetic example · Reviewed
View case
TYPHOON-003
Typhoon
5 / 5
3,000
Synthetic example · Reviewed
View case
EARTHQUAKE-001
Earthquake
2 / 5
200
Synthetic example · Reviewed
View case
EARTHQUAKE-002
Earthquake
4 / 5
1,200
Synthetic example · Reviewed
View case
EARTHQUAKE-003
Earthquake
5 / 5
2,200
Synthetic example · Reviewed
View case
Typhoon Ursula (2019)
Typhoon
4 / 5
1,000
Historical reference · Illustrative plan · Reviewed
View case
Typhoon Odette (2021)
Typhoon
4 / 5
1,800
Historical reference · Illustrative plan · Reviewed
View case
11 of 11 cases


Expected flooding
Available resources
Shelter places (people)
Response teams
3
2
Find similar cases
Use selected case
Notification Center 4
Review plan
From Typhoon Ursula (2019)
Adaptation details
Planning priority
Total shelter places
Total response teams
High
340
3
Additional needed: 337 shelter places · 1 response teams.
Evacuation strategy
For this urban typhoon-and-flood exercise, review early evacuation of flood-prone areas, accessible transport, and routes outside projected flood zones with the LGU.
Resource allocation
Allocate the proposed response teams across evacuation support, relief distribution, and coordination. Confirm transport and supply needs locally.
Shelter capacity plan
Review the proposed shelter capacity and additional places needed against the affected population and confirm shelters are outside projected flood zones.
Review actions
Review projected urban flooding with the LGU coordination team.
Review accessible evacuation transport and shelter registration.
Confirm supply and staffing gaps before approving the exercise plan.
1–10 actions, up to 300 characters each.
Review note
I reviewed this plan.
Save reviewed plan